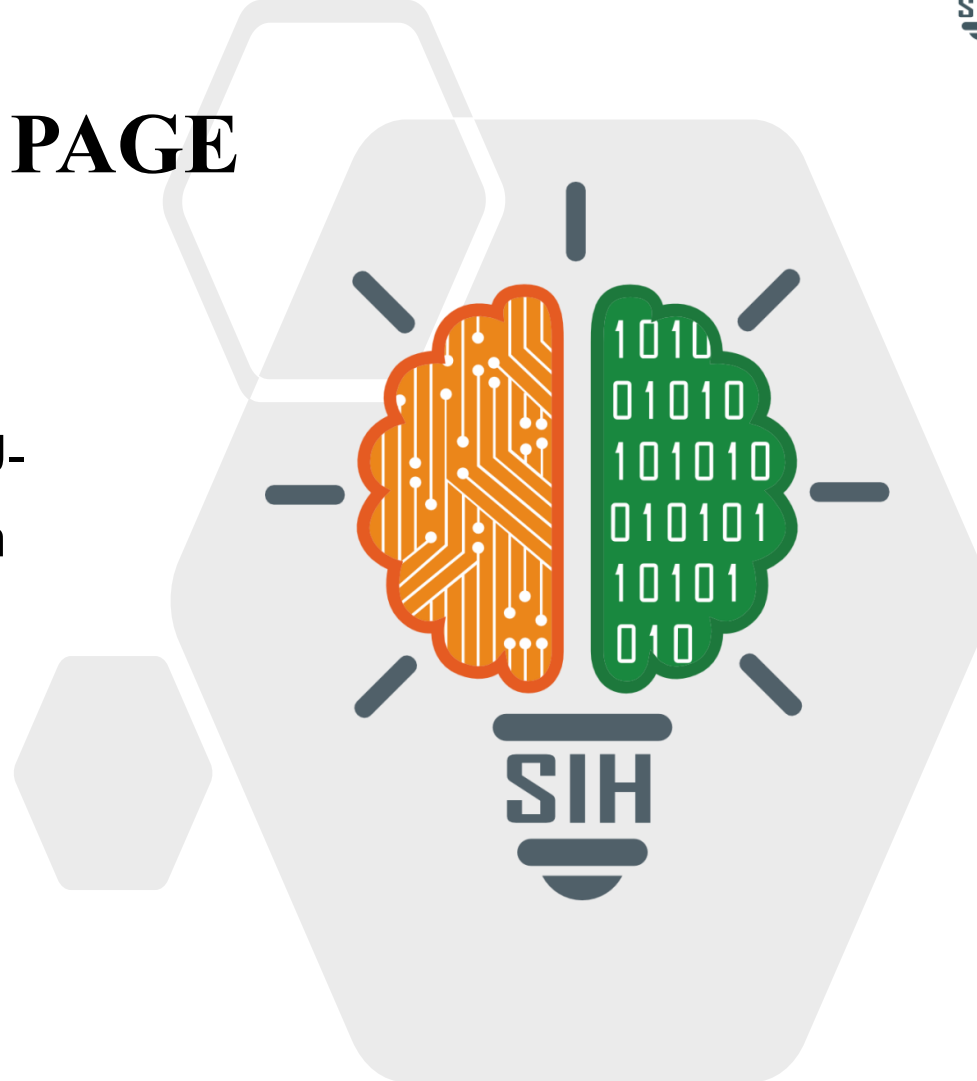


TITLE PAGE

- **Problem Statement ID** – SIH26119
- **Problem Statement Title**- Indigenous GPU-Accelerated Optimization Solver (Sovereign Alternative to Express / CPLEX)
- **Theme**- Smart Automation
- **PS Category**- Software
- **Team ID**- 127364
- **Team Name (Registered on portal)**- VIGHNAX



SAMADHAN: a sovereign, GPU-first optimization solver built from scratch

❖ Proposed Solution (Describe your Idea/Solution/Prototype)

❖ Detailed explanation of the proposed solution

- **Own solver core, no borrowed engine:** MPS reader, scaling and every algorithm written by us; no CPLEX, Gurobi, Xpress, HiGHS or CBC code inside
- **GPU engine for LP and QP (working):** restarted primal-dual hybrid gradient; needs only sparse matrix-vector products, so it runs fully on an NVIDIA GPU
- **C++ core for MILP (working):** dual simplex + branch-and-cut with Gomory cuts; proves 26 MIPLIB 3 models optimal, 3 where HiGHS timed out

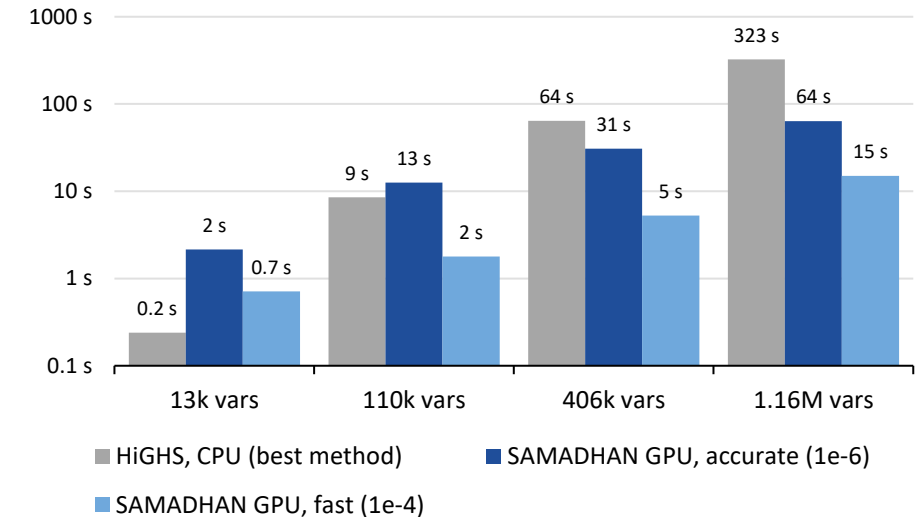
❖ How it addresses the problem

- **Built for refinery planning:** solves crude-blending and product-dispatch LPs up to 1.2 million variables on a laptop GPU
- **Faster where it matters:** on the largest model, $5.1 \times$ faster than HiGHS with cost within 0.00016% of optimal; $22 \times$ faster in fast mode
- **Convex cost curves too (QP):** a 110k-variable refinery QP solved in 1.5 s on the GPU; the HiGHS QP solver had not finished after 600 s

❖ Innovation and uniqueness of the solution

- **GPU-first, factorisation-free:** memory-light and massively parallel, scales past the limits of simplex and interior point
- **Transparent and tunable:** open, auditable code that Indian engineers can inspect and adapt to MRPL's own models

Solve time, refinery planning LP (log scale)



Working prototype; same models, same laptop. Each solver credited with its faster method.

5.1×

faster than HiGHS
on 1.2M variables

22×

faster in fast mode
(1e-4)

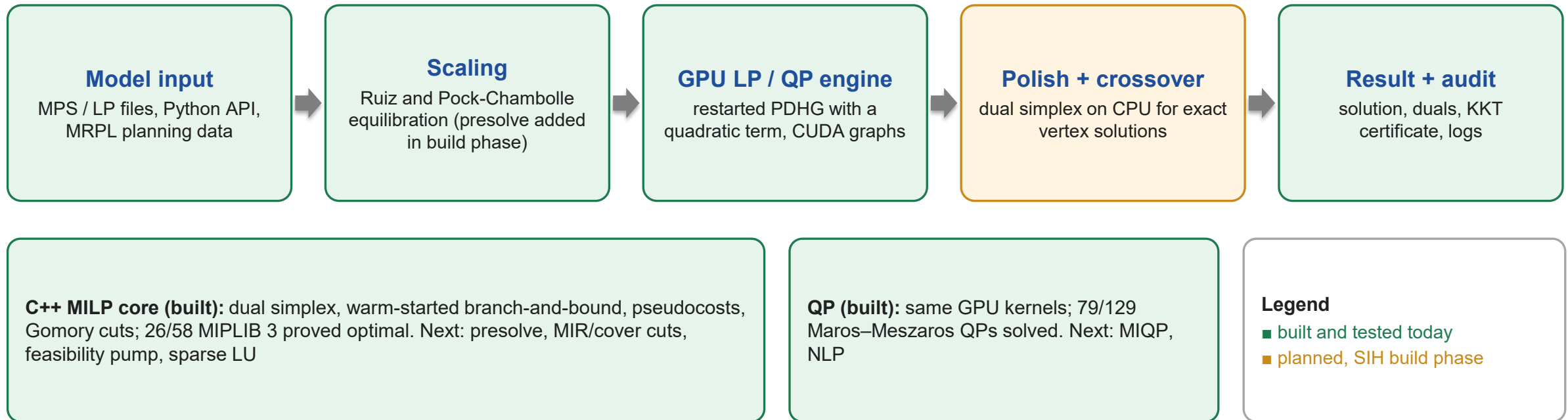
0.00019%

worst cost error vs
optimum (1e-6)

❖ Technologies to be used

- **Solver core:** C++17 (dual simplex, branch-and-cut, built with zig/clang) + PyTorch CUDA sparse kernels and CUDA graphs (GPU LP/QP); Python API on top
- **Interfaces:** Python API, command-line tool, MPS/LP files; REST service for plant planning systems
- **Hardware:** any NVIDIA GPU (tested on RTX 5050 Laptop, 8 GB); the same code runs on CPU-only servers

❖ Methodology and process for implementation



Every stage is our own code. HiGHS is used only as an outside benchmark to check answers and speed.



FEASIBILITY AND VIABILITY



❖ Analysis of the feasibility of the idea

- **Already working:** every refinery test LP solved, cost within 0.00019% of the true optimum
- **Modest hardware:** 1.2M variables, 2.3M nonzeros on one 8 GB laptop GPU
- **Proven, published algorithms:** PDLP-type methods are peer-reviewed and used in commercial solvers
- **Clear plan:** LP, QP and MILP working; presolve, crossover, cuts and scale-up next

Netlib: 86/91 LPs solved to 1e-4 relative KKT (80 within 0.1% of optimum); MPS reader matches HiGHS on 91/91. The 5 left are ill-conditioned cases for our simplex crossover.

MIPLIB 3: 26/58 proved optimal (HiGHS 46; 3 only by us). **QP:** 79/129 Maros–Meszáros solved (HiGHS 99).

Risk	How we handle it
First-order methods give moderate accuracy	Restarts and tight KKT checks, then simplex crossover for exact vertices
Hard MILPs need years of tuning	HiGHS still proves 20 more MIPLIB models; add presolve, cuts and heuristics step by step
Degenerate or ill-conditioned models	Ruiz scaling, float64, presolve; crossover for Netlib's hard cases (pilot4, perold, greenbea)
No GPU at a site	Same code runs on CPU; GPU only where it is measurably faster

Refinery LP	Variables	Nonzeros	HiGHS (best)	Accurate 1e-6	Cost error	Speed-up	Fast 1e-4	Cost error	Speed-up
Size S	13,296	25,416	0.2 s	2.2 s	0.000069%	0.1×	0.7 s	0.013%	0.3×
Size M	109,536	214,092	8.5 s	12.6 s	0.00019%	0.7×	1.8 s	0.017%	4.8×
Size L	406,080	796,512	63.9 s	30.7 s	0.000035%	2.1×	5.3 s	0.015%	12×
Size XL	1,157,120	2,273,888	323 s	63.7 s	0.00016%	5.1×	15.0 s	0.070%	22×

Measured today on one laptop (RTX 5050 8 GB GPU). HiGHS: faster of dual simplex and interior point, all CPU threads. SAMADHAN: faster of its two GPU methods, at 1e-6 or 1e-4 relative KKT. Cost error vs HiGHS optimum.

❖ Potential impact on the target audience

- **MRPL and other refineries:** faster crude selection, blending and dispatch plans; more what-if scenarios per planning cycle
- **Power, logistics, steel, railways:** the same engine serves unit commitment, routing and production planning
- **Strategic users:** runs air-gapped and on-premise, with no licence server and no data leaving the site

❖ Benefits of the solution (social, economic, environmental, etc.)

Economic

No per-core or per-user licence fees;
GPU speed-ups cut planning time

Sovereignty

Indian-owned solver code that can be
audited, changed and certified

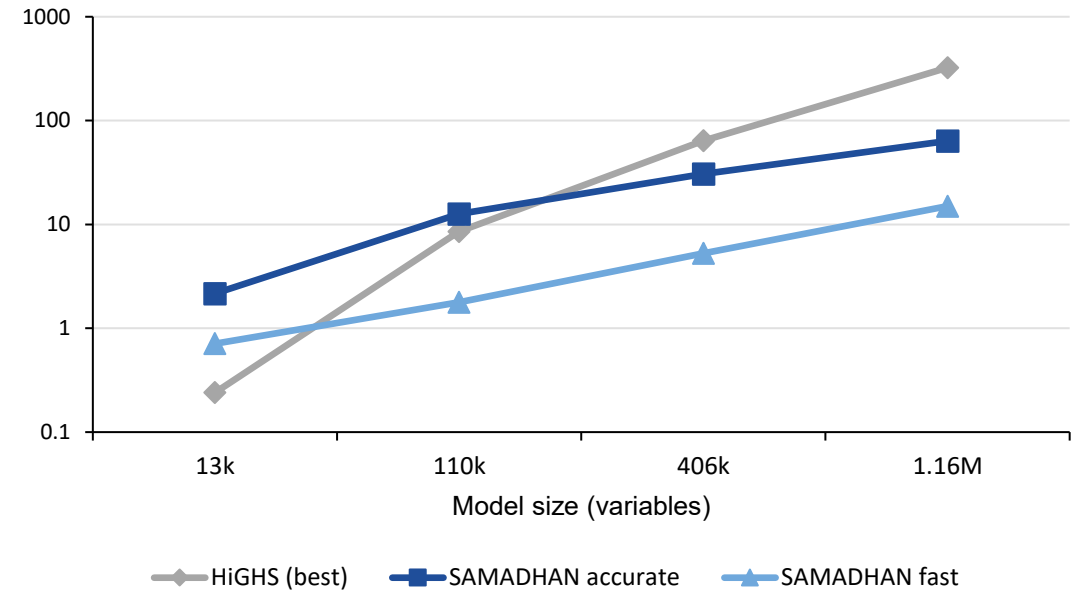
Environmental

Better crude and energy plans mean
less fuel, flaring and waste

Social

Builds Indian skills in optimisation
software, a rare capability

Solve time vs model size (seconds, log scale)



Refinery planning LPs from 13k to 1.16M variables. The gap widens with size, which is where real plant-wide models live.

Every licence we replace is money saved each year, and every model we solve in-house stays under Indian control.

❖ Details / Links of the reference and research work

❖ LP on GPUs (our core)

- **PDLP:** Applegate, Díaz, Hinder, Lu, Lubin, O'Donoghue, Schudy, Practical Large-Scale Linear Programming using Primal-Dual Hybrid Gradient, NeurIPS 2021
- **cuPDLP:** Lu & Yang, cuPDLP.jl: A GPU Implementation of Restarted PDHG for Linear Programming, arXiv:2311.12180 (2023)
- **PDHG and scaling:** Chambolle & Pock, J. Math. Imaging Vision 40 (2011); Ruiz, A scaling algorithm to equilibrate both rows and columns norms in matrices, RAL-TR-2001-034

❖ MILP and QP

- **Branch-and-cut:** Achterberg, Constraint Integer Programming, PhD thesis, TU Berlin (2007); Bixby, A Brief History of Linear and Mixed-Integer Programming Computation, Doc. Math. (2012)
- **Dual simplex:** Huangfu & Hall, Parallelizing the dual revised simplex method, Math. Prog. Comp. 10 (2018)
- **QP:** Lu & Yang, A Practical and Optimal First-Order Method for Large-Scale Convex QP (PDQP), arXiv:2311.07710; Stellato et al., OSQP, Math. Prog. Comp. 12 (2020)

❖ Benchmarks the PS asks for

- **MIPLIB 3 / 2017:** Bixby et al., An updated mixed integer programming library: MIPLIB 3.0, Optima 58 (1998); Gleixner et al., Math. Prog. Comp. 13 (2021)
- **Maros–Mészáros QP set:** Maros & Mészáros, A repository of convex QP problems, Optim. Methods Softw. 11 (1999)
- **Netlib LP and Mittelman benchmarks:** netlib.org/lp • plato.asu.edu/bench.html



Source code and benchmarks

github.com/NAMAN-THAKUR-1944/
SIH2026-SIH26119-SAMADHAN

Reproduce every number

```
python -m pytest
python bench_final.py    (LP)
python bench_netlib.py   (Netlib)
python bench_miplib.py   (MIPLIB 3)
python bench_qp.py maros (QP)
```